

Mikroelektronische Schaltungen und Systeme

Vorlesung 17 Technologie Mapping

Prof. Jürgen Becker
Institut für Technik der Informationsverarbeitung (ITIV)



1. Einführung Graphen
2. Cell-Library Binding
3. Structural Matching
4. Polarity Assignment Problem
5. Rule-Based Library Binding
6. Zieltechnologie CMOS



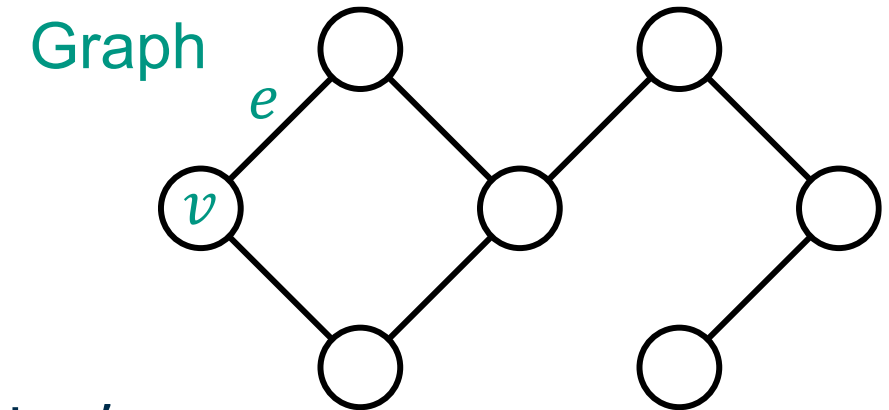
Einführung Graphen

1

Einführung Graphen

Graphs

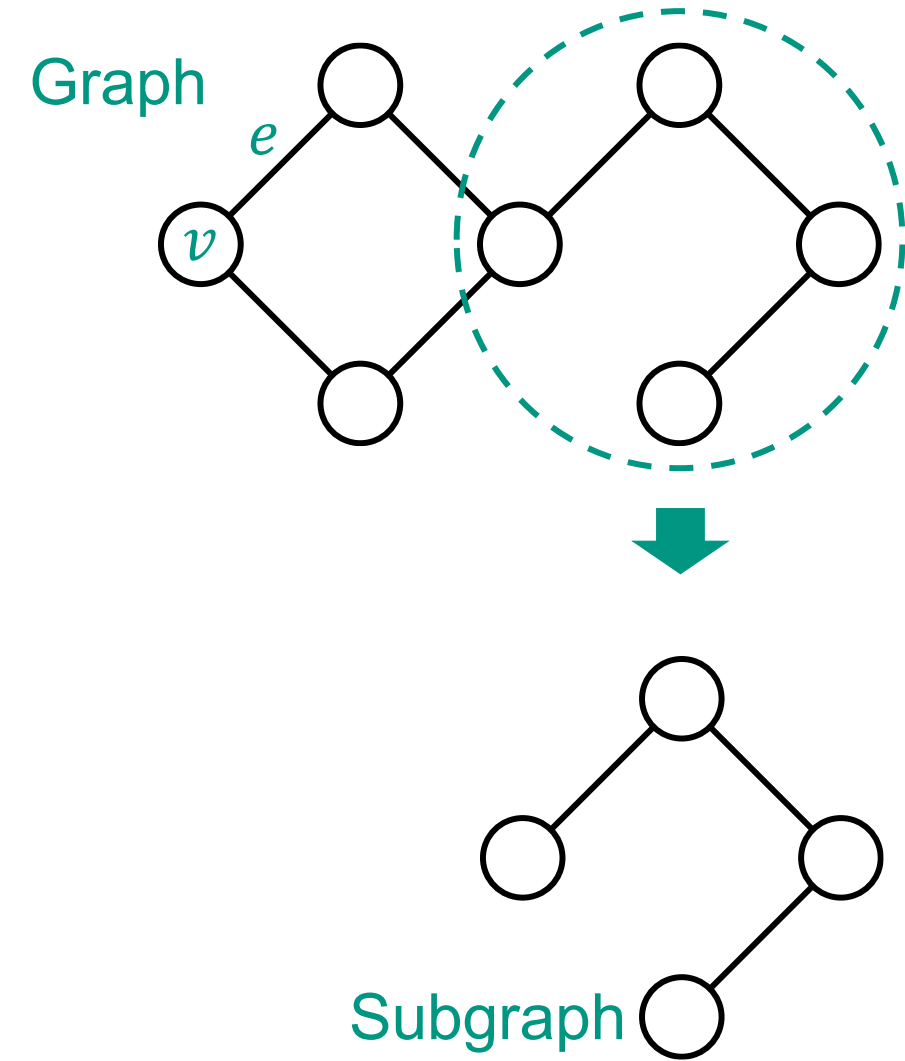
- Ein Graph $G = (V, E)$ besteht aus
 - Set von Vertex/Vertices (Nodes) V
 - Set von Edges $E \subseteq V \times V$
 - Ein Paar $(v, v') \in E$ wird als Edge e von v nach v' bezeichnet.
 - Eine Edge e ist entweder **directed** oder **undirected**.
 - Verbundene Vertices v und v' werden als **adjacent** bezeichnet. Sie werden als **incident** mit der verbindenden Edge bezeichnet.



Einführung Graphen

Subgraphs

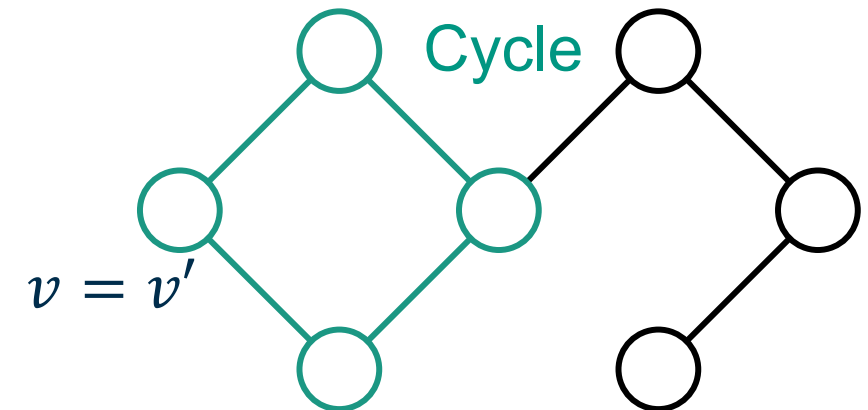
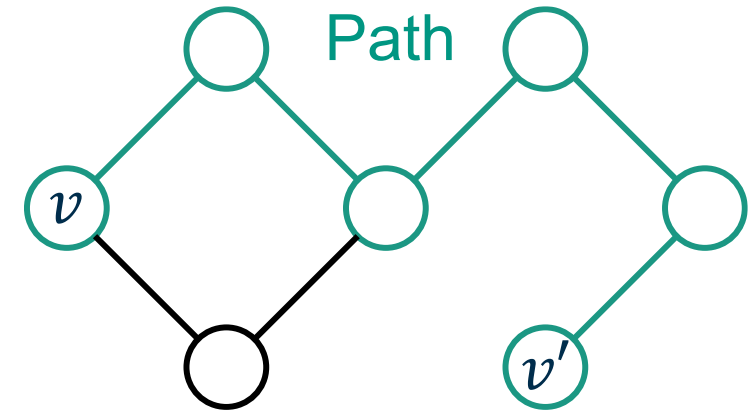
- Ein Graph $G' = (V', E')$ ist ein **Subgraph** von $G = (V, E)$ wenn $V' \subseteq V$ und $E' \subseteq E$.



Einführung Graphen

Paths und Cycles

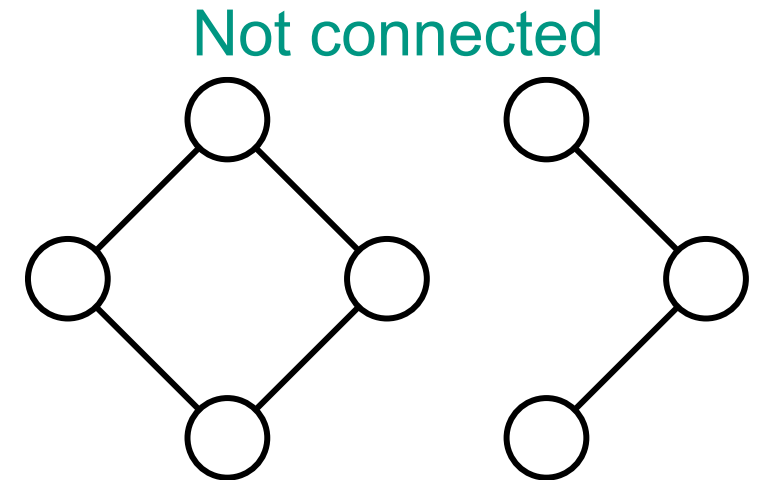
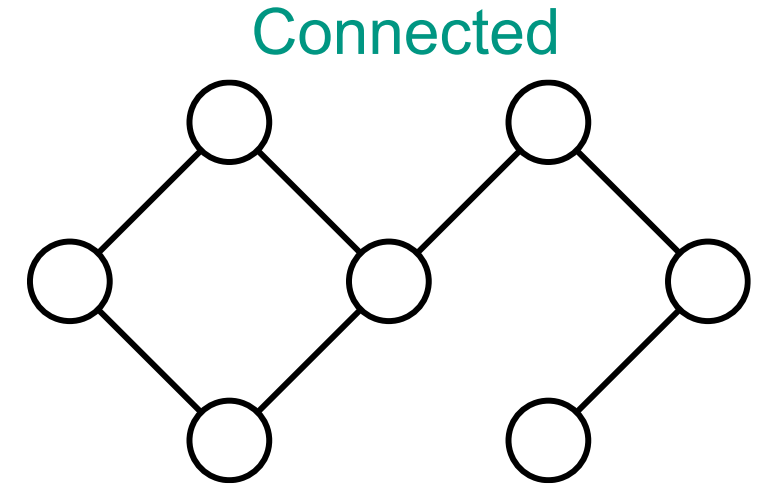
- Ein **Path** ist ein Subgraph eines Graphen von v bis v' der durch eine Folge $(v_0, v_1, \dots, v_k)$ mit $v_0 = v, v_k = v'$ beschrieben wird.
 - Ein Pfad wird als **simple** bezeichnet, wenn kein Knoten mehr als einmal besucht wird.
- Ein **Cycle** ist ein Path mit $v = v'$ d. h. ein Pfad, der am Start-Vertex endet.



Einführung Graphen

Connected Graphs

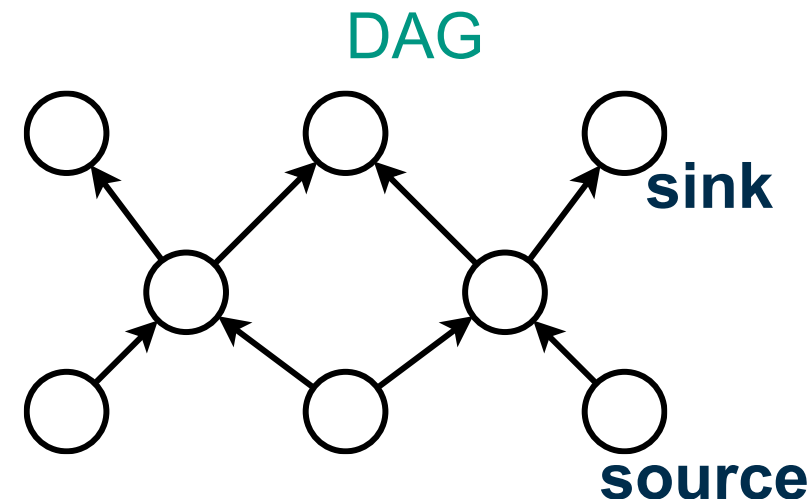
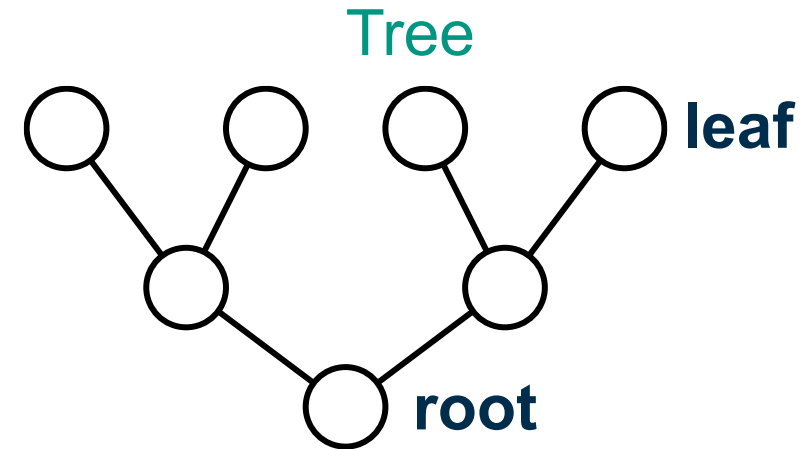
- Ein Graph ist **connected** wenn für alle Paare von vertices (v, v') im Graphen ein Path existiert.



Einführung Graphen

Trees und DAGs

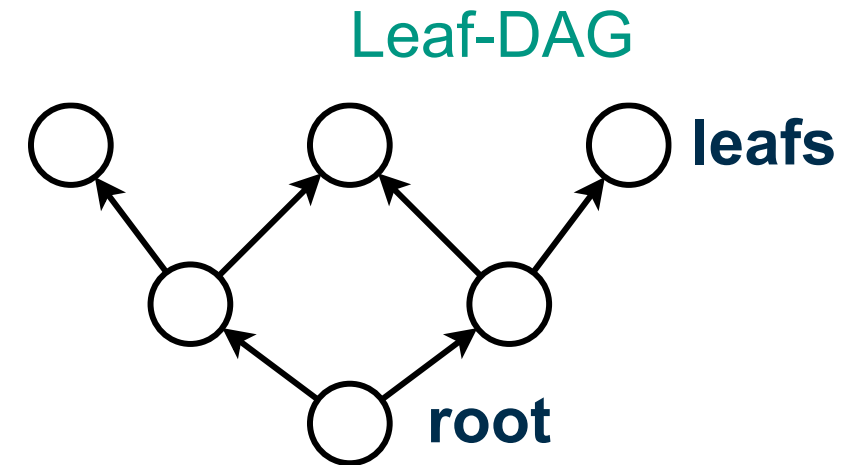
- Ein **Tree** ist ein **connected Graph** ohne Cycle.
 - Ein **Leaf** ist ein Vertex mit dem Grad 1.
 - Ein **Rooted Tree** ist ein Tree mit einem bestimmten Vertex, der als Root bezeichnet wird. Jede Edge führt implizit von Root weg.
- Ein **DAG** (**directed acyclic graph**) ist ein **directed graph** ohne Cycle.
 - Ein Vertex ohne Nachfolger wird als **sink** bezeichnet.
 - Ein Vertex ohne Vorgänger wird als **source** bezeichnet.



Einführung Graphen

Leaf-DAG

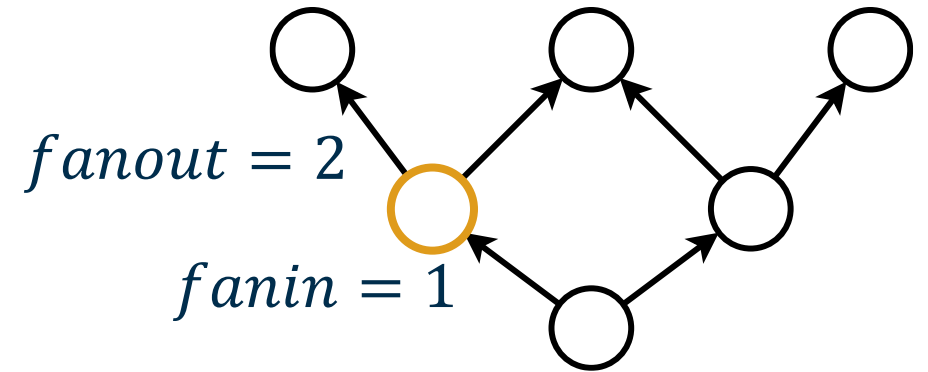
- Der Begriff **leaf-DAG** wird für **rooted DAGs** (nur ein Source-Vertex) verwendet, bei denen Pfade, die von Root ausgehen, nur an den Sink-Vertices (Leafs) wieder zusammenlaufen können.



Einführung Graphen

Fanin und Fanout

- **Fanin** ist die Anzahl der eingehenden edges.
- **Fanout** ist die Anzahl der ausgehenden edges.





Cell-Library Binding

2

Cell-Library Binding

Einführung

Definition

Das **Cell-Library Binding** ist die Aufgabe, eine **ungebundene** allgemeine Logikschaltung mit einer **gebundenen** Logikschaltung abzubilden, die mit Instanzen vordefinierter Elemente aus einer bestimmten Bibliothek implementiert ist.

- Die gegebene Bibliothek ist **technologieabhängig**.
- Cell-Library Binding wird auch als **Technology Mapping** bezeichnet.



Cell-Library Binding

Einführung

- Die **technologischen Parameter** werden durch die Charakterisierung der Bibliothek abgebildet.
- Eine Zellbibliothek besteht aus einer Sammlung **primitiver Logik-Zellen** mit:
 - Boolescher Funktion
 - Terminals (Anschlüsse für Eingabe/Ausgabe)
 - Delay Eigenschaften
 - Fläche
 - Kapazitive Last
- Der **Binding Prozess** muss die Eigenschaften der Bibliothek nutzen, um die bestmögliche **optimale Implementierung** zu finden.

Cell-Library Binding

Einführung

- Ein gängiger Ansatz für das Library Binding besteht darin, das Binding auf den Ersatz von **Subnetzwerken** durch Instanzen aus der Zellenbibliothek zu beschränken.
- Dies ist das sogenannte **Network Covering Problem**.
- Das **Covering** umfasst die Identifizierung von Teilen des logischen Netzwerks, die durch eine Bibliothekszelle ersetzt werden können, das Covering des gesamten Netzwerks und die Optimierung in Bezug auf bestimmte Parameter.
- Eine Zelle **matcht** ein Subnetzwerk, wenn die boolesche Funktion äquivalent ist.

Cell-Library Binding

Match Methods

- Boolean Match
 - Boolean Matching kann durch Überprüfung der Äquivalenz zwischen zwei booleschen Funktionen verifiziert werden.
 - Structural Match
 - Structural Matching kann durch Überprüfung der Isomorphie zwischen zwei rooted DAGs überprüft werden.
- Ein Structural Match impliziert ein Boolean Match, aber nicht andersherum.
- Beide beschriebenen Methoden setzen vorheriges **decomposition und partitioning** voraus.

Cell-Library Binding

Trivial Binding

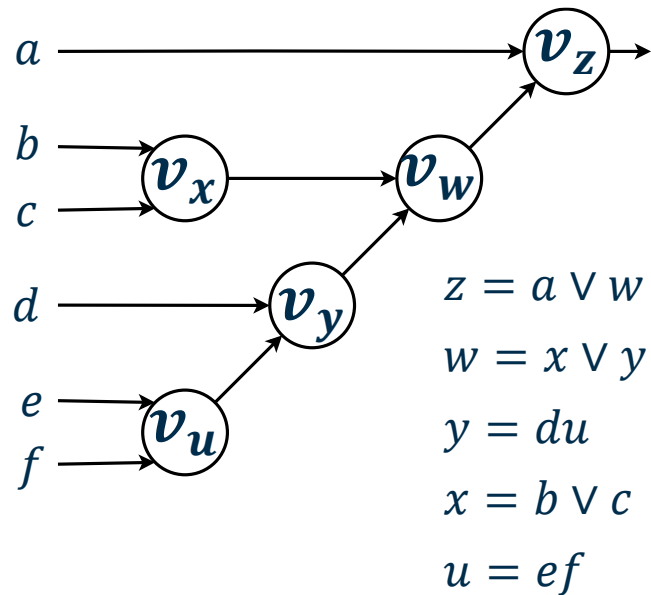
- Die Bibliothek muss Zellen enthalten, die die **booleschen Basisfunktionen** implementieren, um die Existenz einer Lösung sicherzustellen.
- **Trivial Binding** ist der Prozess der Übersetzung eines ungebundenen Netzwerks in ein gebundenes Netzwerk, indem jeder vertex einzeln an eine passende Bibliothekszelle gebunden wird.
- Trivial Binding ist möglicherweise nicht optimal, selbst wenn das ungebundene Netzwerk optimal ist, da ungebundene Logiknetzwerke ohne Berücksichtigung der Technologie und der Zellparameter optimiert werden.



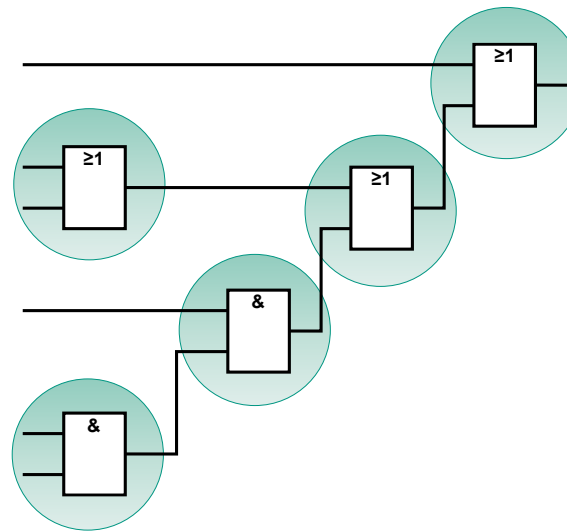
Cell-Library Binding

Network Covering

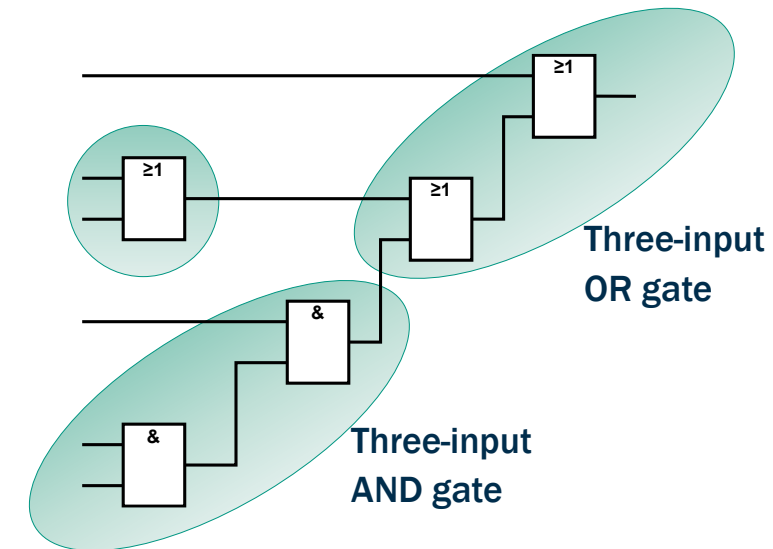
Beispiel



Unbound Network



Bound Network
(Trivial Binding)



Bound Network
(Library mit 3-input gates)

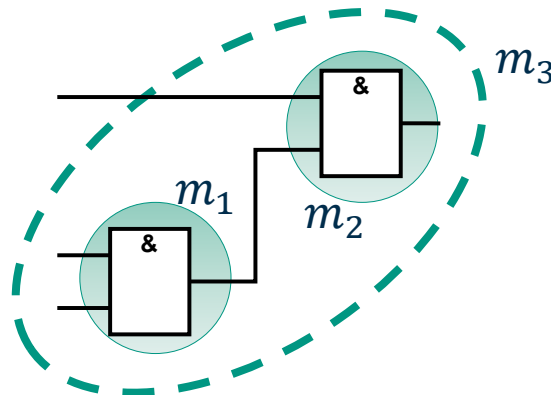
Cell-Library Binding

Network Covering

Anmerkung

- Jeder interne **Vertex** muss durch mindestens eine Übereinstimmung gedeckt sein.
 - Für jedes Match müssen alle Input- und Outputanschlüsse entweder mit anderen matched Zellen verbunden oder mit den Außenanschlüssen des Netzwerks assoziiert sein.
- Die Wahl eines Matches kann also die Wahl anderer Matches implizieren.

Beispiel



Constraints:

$(m_1 \vee m_3)$ *vertex_1 is covered*

$(m_2 \vee m_3)$ *vertex_2 is covered*

$(\overline{m_2} \vee m_1)$ *match m_2 implies m_1*

$\Rightarrow (m_1 \vee m_3)(m_2 \vee m_3)(\overline{m_2} \vee m_1) = 1$ *must hold*

Cell-Library Binding

Ansätze

- Praktische Ansätze zum Cell-Library Binding lassen sich in zwei Hauptgruppen einteilen:
 - **Heuristische Algorithmen**
 - Nähert sich der besten Lösung des Network Covering Problems an.
 - Findet schneller eine Lösung, jedoch meist nicht die optimale Lösung.
 - **Regelbasierte Ansätze**
 - Das Netzwerk wird durch schrittweise Verfeinerung gebunden. Eine Reihe lokaler Transformationen optimierte das gebundene Netzwerk in Bezug auf Parameter.

Cell-Library Binding

Decomposition und Partitioning

- Die meisten heuristischen Algorithmen wenden zwei Vorverarbeitungsschritte an:
 - **Decomposition**
 - Ist erforderlich, um eine Lösung zu garantieren.
 - Alle booleschen Funktionen in **Basisfunktionen** transformieren, die von der Zellenbibliothek abgedeckt werden.
 - Dadurch wird sichergestellt, dass zumindest das Trivial Binding möglich ist.
 - **Partitioning**
 - Wandeln Sie ein multiple-input multiple-output Netzwerk in eine Sammlung von multiple-input single-output Netzwerken um. Diese **Subnetzwerke** werden als **Subject Graphs** bezeichnet.
 - Die Größe jedes Abdeckungsproblems nimmt ab.
 - Zusätzliche Annahmen können auf Grundlage von **Trees** oder **leaf-DAGs** getroffen werden.

Cell-Library Binding

Decomposition und Partitioning

- Die meisten heuristischen Algorithmen basieren auf:
 - **Decomposition**
 - Ist erforderlich, um eine Lösung zu finden.
 - Alle booleschen Funktionen in Basisfunktionen zerlegt werden müssen, die in der Bibliothek abgedeckt werden.
 - Dadurch wird sichergestellt, dass zumindest das Trivial Binding möglich ist.
 - **Partitioning**
 - Wandeln Sie ein multiple-input multiple-output Netzwerk in ein Set von multiple-input single-output Netzwerken um. Diese **Subnetzwerke** werden als **Subject Graphs** bezeichnet.
 - Die Größe jedes Abdeckungsproblems nimmt ab.
 - Zusätzliche Annahmen können auf Grundlage von **Trees** oder **leaf-DAGs** getroffen werden.

multiple-input single-output Subnetzwerken

$\triangleq$

Subject Graph

**Die nachfolgenden Matching Algorithmen
legen Subject Graphen zugrunde.**



Structural Matching

3

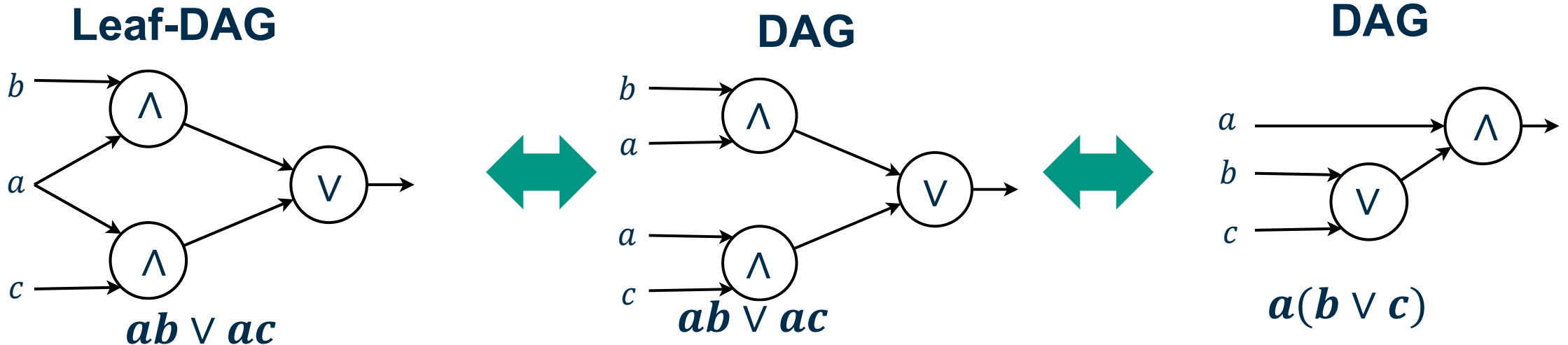
Structural Matching

Partitioning

Anmerkung

Jeder **Leaf-DAG** kann leicht in einen rooted DAG-Graphen umgewandelt werden, indem die Blätter des entsprechenden leaf DAGs aufgeteilt werden.

Beispiel



Structural Matching

Pattern Graphen

- Die mit Bibliothekszellen assoziierten Graphen werden als **Pattern Graphs** bezeichnet.
- Pattern Graphen sind Graphen, die von **structural matching Algorithmen** verwendet werden, um Übereinstimmungen mit Subgraphen von Subjekt Graphen zu finden.

Anmerkung

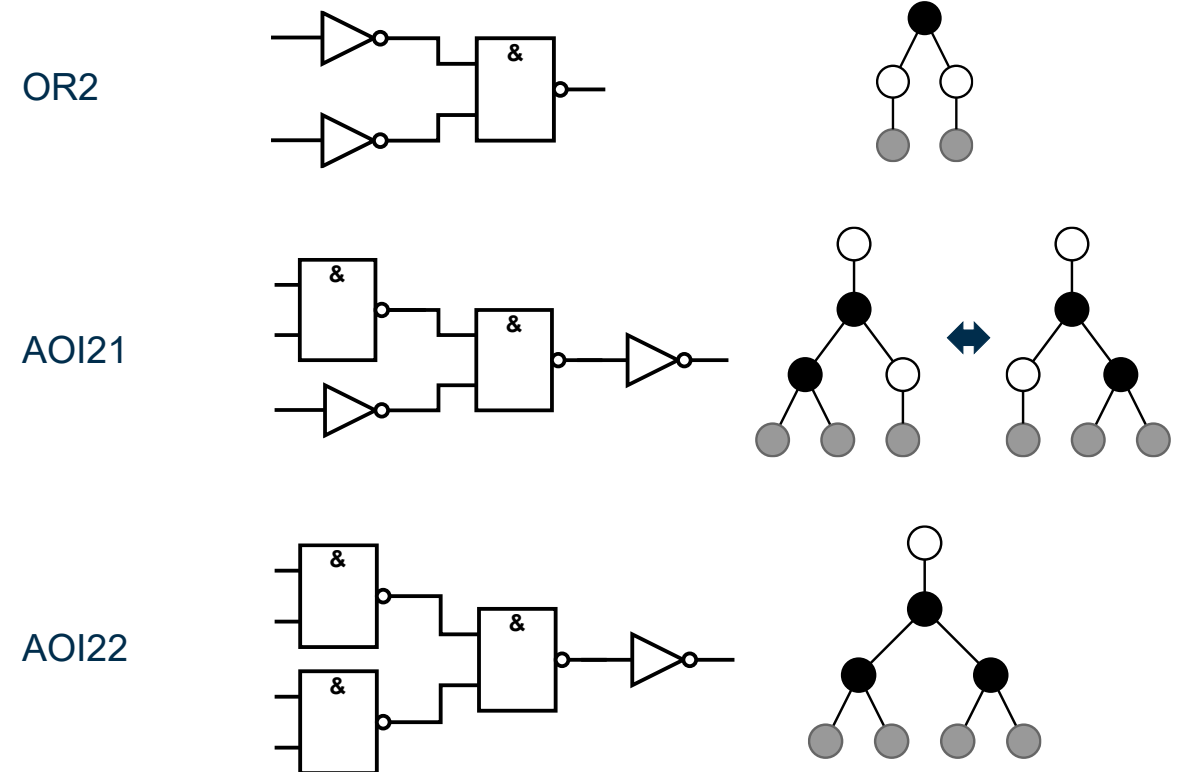
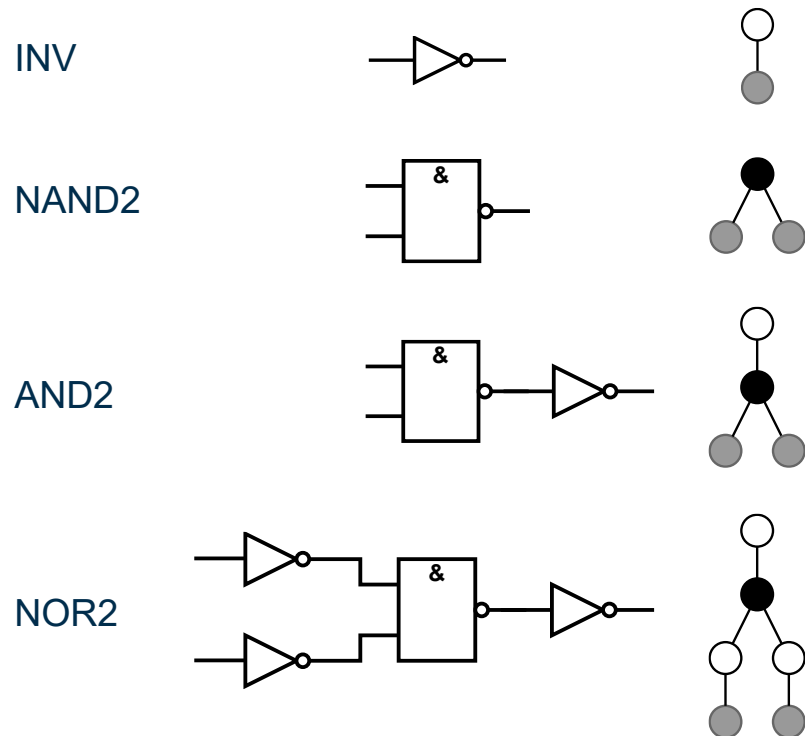
Einige Bibliothekszellen haben **mehr als einen** assoziierten Pattern Graphen, da die Decomposition der repräsentierten booleschen Funktion nicht einzigartig ist.

Structural Matching

Structural Matching

Beispiel

(Nand2 / INV) base system

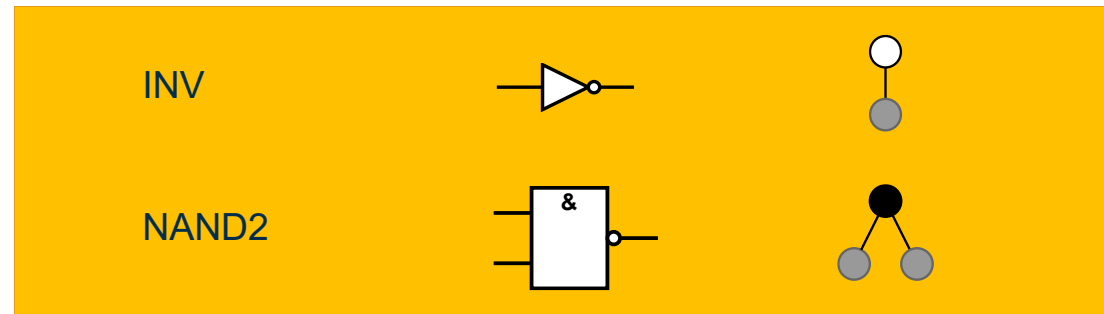


Structural Matching

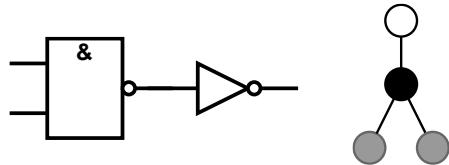
Structural Matching

Beispiel

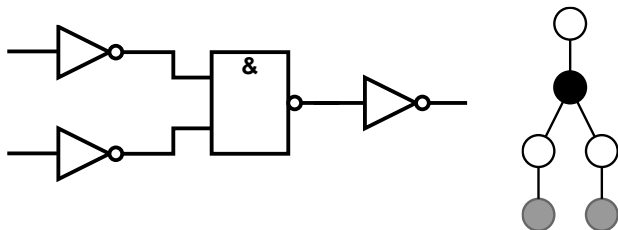
(Nand2 / INV) base system



AND2

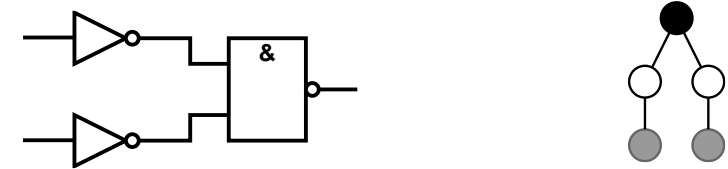


NOR2

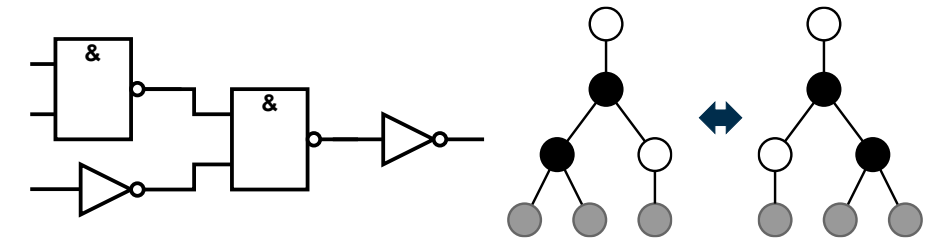


Base System

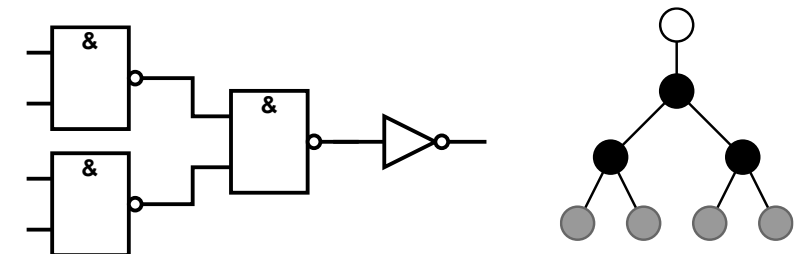
OR2



AOI21



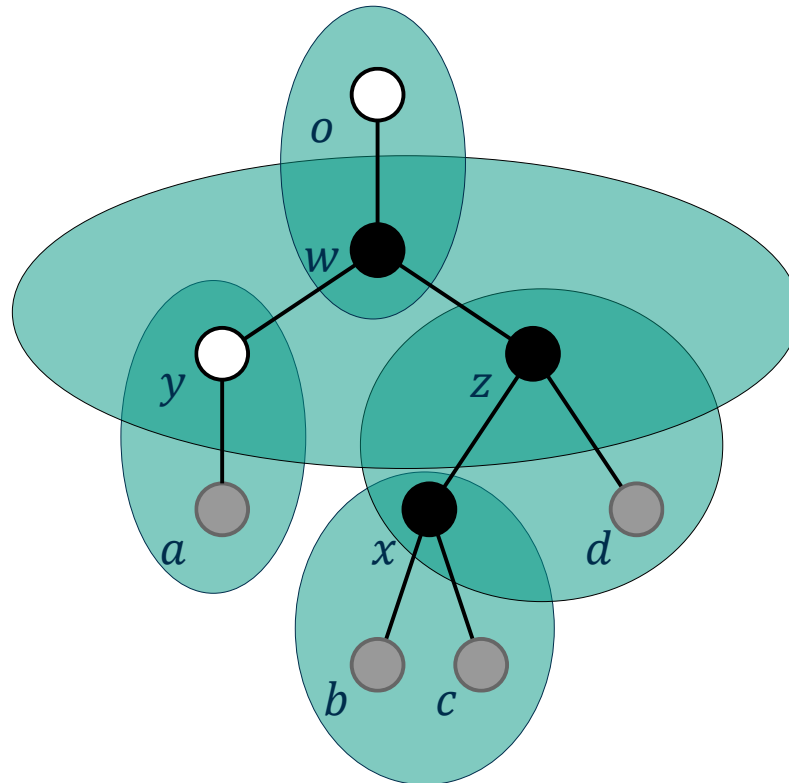
AOI22



Structural Matching

Tree-Based Covering

- Die **Tree-Based Covering** Methode nutzt das Structural Matching um einen Tree (Subject Graph) im bottom-up Ansatz zu traversieren und zu covern.



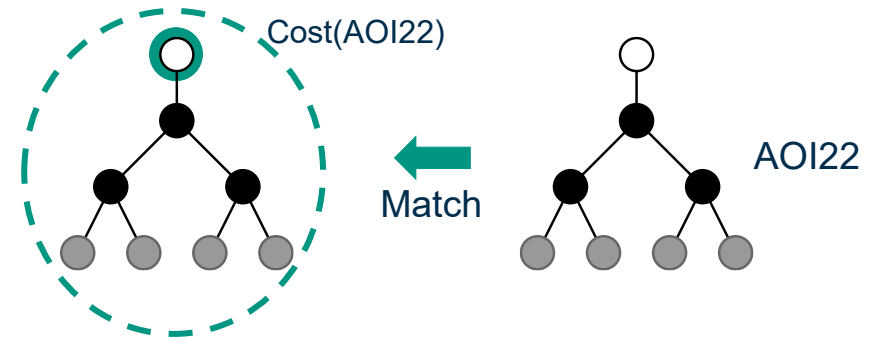
Structural Matching

Tree-Based Covering

■ Dynamic Programming

- Für alle Knoten des Subject Graphen ermittelt der Covering Algorithmus die Übereinstimmungen der **locally Rooted Subtrees** mit den Pattern Graphen. Für jeden gegebenen Pattern Tree gibt es drei Möglichkeiten:

1. Der Pattern Tree und der locally Rooted Subtree sind isomorph. Anschließend wird der vertex mit den entsprechenden Zellkosten beschriftet.



● : Root Vertex

○ : Locally Rooted Subtree

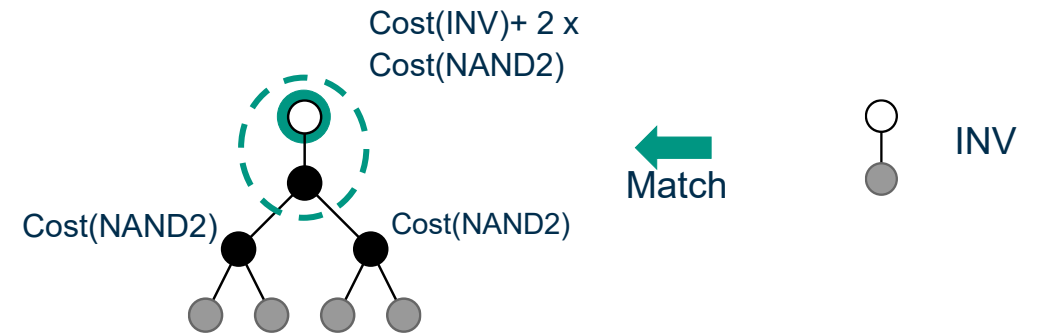
Structural Matching

Tree-Based Covering

■ Dynamic Programming

- Für alle Knoten des Subject Graphen ermittelt der Covering Algorithmus die Übereinstimmungen der locally Rooted Subtrees mit den Pattern Graphen. Für jeden gegebenen Pattern Tree gibt es **drei Möglichkeiten**:

2. Der Pattern Tree ist isomorph zu einem Subtree des locally rooted subject subtree mit demselben Root und einer Menge von leaves L. Dann wird der Knoten mit den entsprechenden Zellkosten plus den Kosten der vertices L beschriftet.



● : Root Vertex

○ : Locally Rooted Subtree

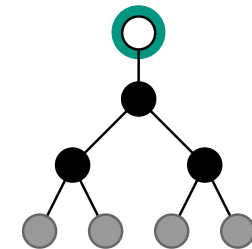
Structural Matching

Tree-Based Covering

- Dynamic Programming

- Für alle Knoten des Subject Graphen ermittelt der Covering Algorithmus die Übereinstimmungen der locally Rooted Subtrees mit den Pattern Graphen. Für jeden gegebenen Pattern Tree gibt es **drei Möglichkeiten**:

3. Es gibt kein Match.



←
No Match

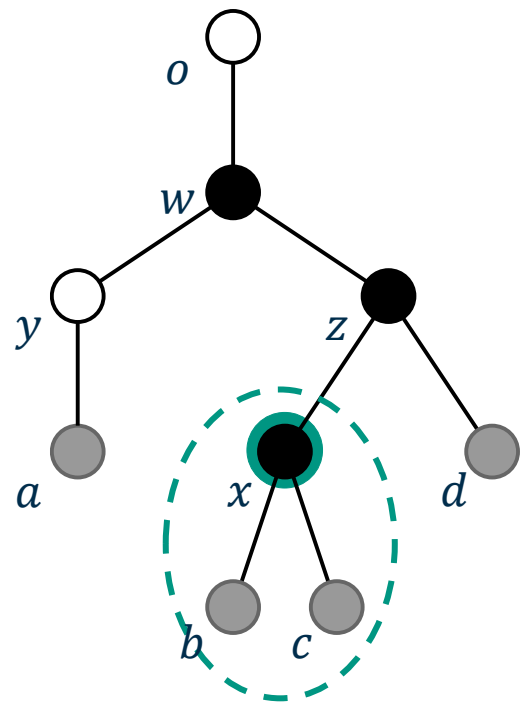
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y			
z			
w			
o			

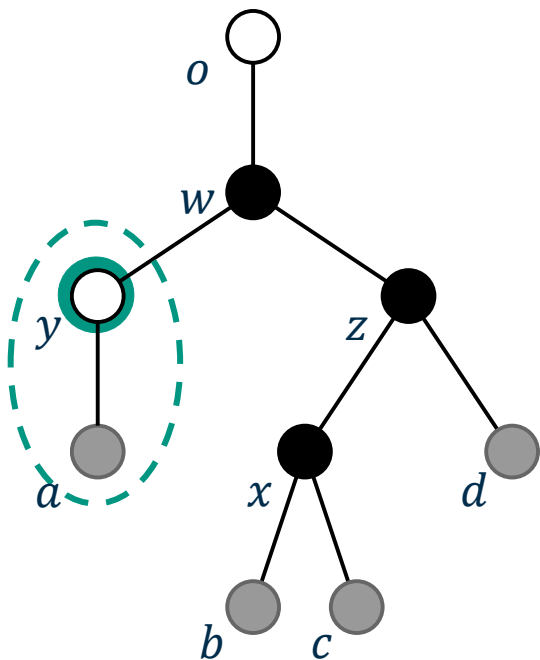
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z			
w			
o			

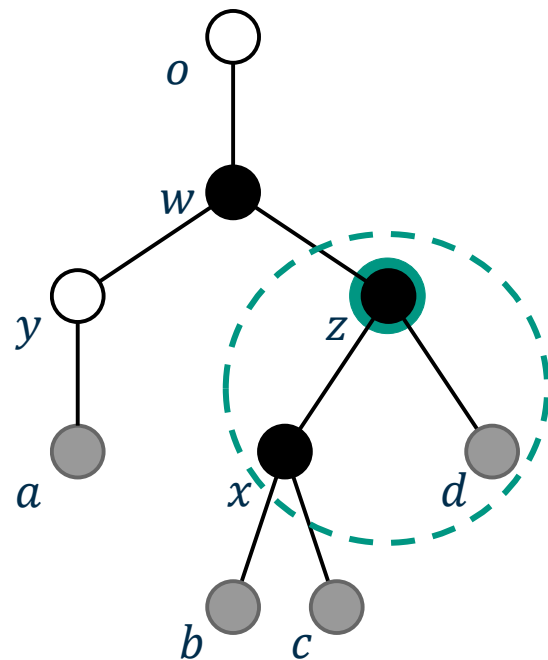
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z	NAND2	$NAND2(x, d)$	$2\ NAND2$
w			
o			

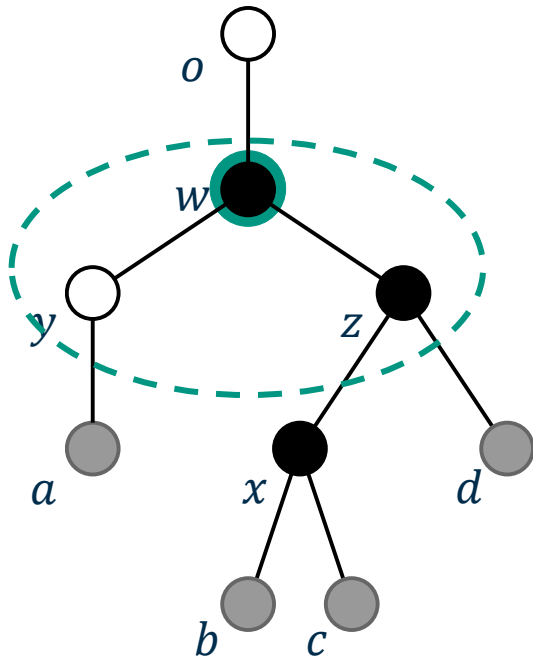
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z	NAND2	$NAND2(x, d)$	2 NAND2
w	NAND2	$NAND2(y, z)$	$3 \text{ NAND2} + INV$
o			

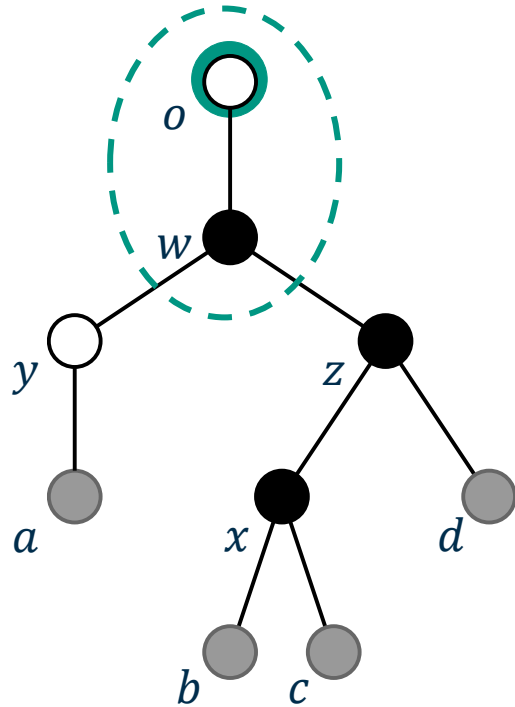
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z	NAND2	$NAND2(x, d)$	2 NAND2
w	NAND2	$NAND2(y, z)$	$3 \text{ NAND2} + INV$
o	INV	$INV(w)$	$3 \text{ NAND2} + 2 \text{ INV}$

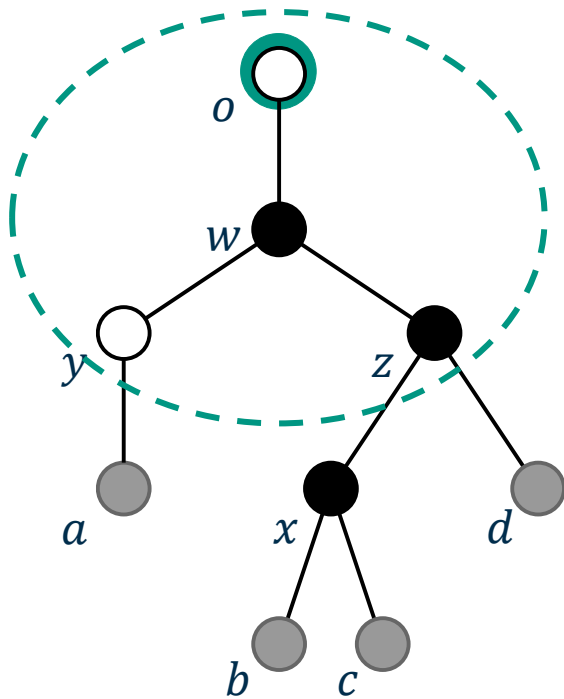
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z	NAND2	$NAND2(x, d)$	2 NAND2
w	NAND2	$NAND2(y, z)$	$3 \text{ NAND2} + INV$
o	INV	$INV(w)$	$3 \text{ NAND2} + 2 \text{ INV}$
	AND2	$AND2(y, z)$	$2 \text{ NAND2} + AND2 + INV$

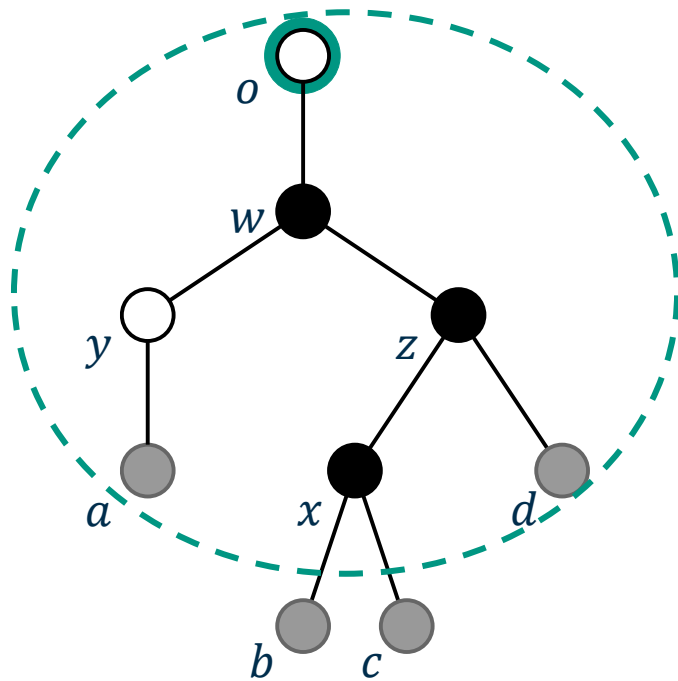
● : Root Vertex

○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z	NAND2	$NAND2(x, d)$	2 NAND2
w	NAND2	$NAND2(y, z)$	$3 \text{ NAND2} + INV$
o	INV	$INV(w)$	$3 \text{ NAND2} + 2 \text{ INV}$
	AND2	$AND2(y, z)$	$2 \text{ NAND2} + AND2 + INV$
	AOI21	$AOI21(x, d, a)$	$NAND2 + AOI21$

● : Root Vertex

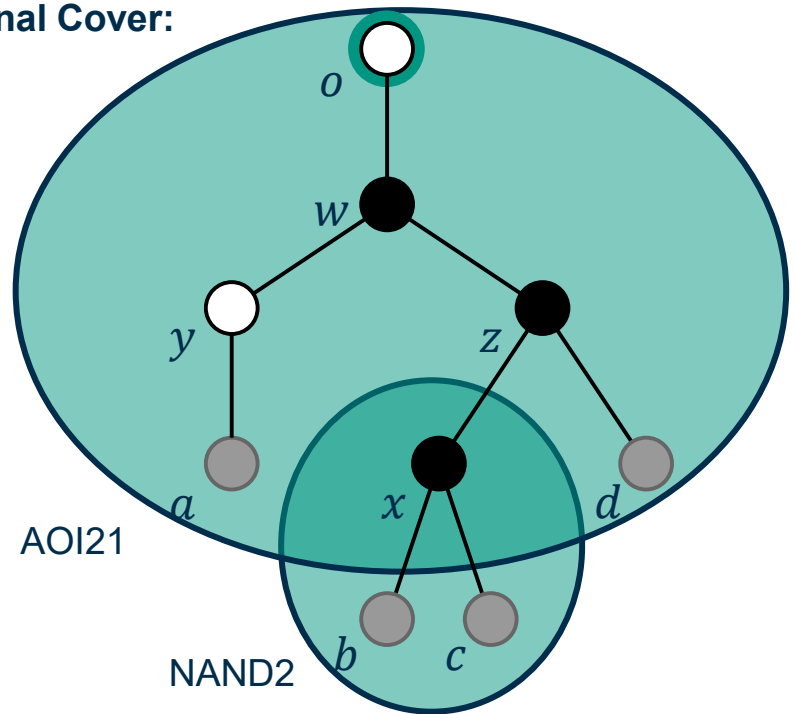
○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Beispiel

Final Cover:



Vertex	Match	Gate	Cost
x	NAND2	$NAND2(b, c)$	$NAND2$
y	INV	$INV(a)$	INV
z	NAND2	$NAND2(x, d)$	$2\ NAND2$
w	NAND2	$NAND2(y, z)$	$3\ NAND2 + INV$
o	INV	$INV(w)$	$3\ NAND2 + 2\ INV$
	AND2	$AND2(y, z)$	$2\ NAND2 + AND2 + INV$
	AOI21	$AOI21(x, d, a)$	$NAND2 + AOI21$

● : Root Vertex ○ : Locally Rooted Subtree

Structural Matching

Tree-Based Covering

Anmerkung

Das tree-based Covering mit dem Dynamic Programming Ansatz ist trotz der Optimalität für Trees heuristisch, da die Bäume nur Subnetzwerke (Subject Graphs) des ursprünglichen Problems sind. Daher ist nicht garantiert, dass das Gesamtergebnis optimal ist.

Structural Matching

Leaf-DAGs

Anmerkung

- Decomposed EXOR und EXNOR Gatter können nicht durch Trees dargestellt werden. Es ist möglich, den tree-covering Algorithmus so zu erweitern, dass er leaf-DAGs verwendet, aber die pattern leaf-DAGs müssen mit den entsprechenden Leafs übereinstimmen. Daher ist nur eine begrenzte Verwendung dieser Zellen möglich.
- Structural Matching erkennt nur eine Teilmenge der möglichen Cover.
- Structural Matching erlaubt die Verwendung von *don't care* Informationen.

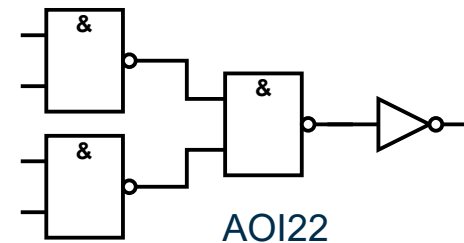
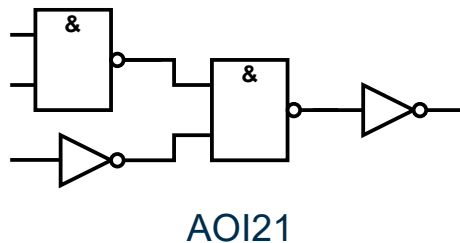
Boolean Matching

Cell Generator Technology Mapping

- Zellgeneratoren sind eine **parametrisierte Zellfamilie** anstelle einer Zellbibliothek.
- Diese Zellfamilie enthält alle Mitglieder einer Funktionsklasse, die die vordefinierten Parameter der Familie nicht überschreiten.
- Aufgrund der **Vollständigkeit** der Zellfamilie kann dies die Zuordnung von Teilnetzwerken zu Zellen vereinfachen, wenn das Teilnetzwerk mit der entsprechenden Zellfamilie übereinstimmt.

Beispiel

AndOrInverters (**AOIs**) (parametrisierbare Anzahl von Eingängen)



Referenz S. 13 [3]



Polarity Assignment Problem

4

Polarity Assignment Problem

Einführung

Definition

Das Problem, ein Subnetzwerk einer Bibliothekszelle zuzuordnen, die die Komplementärfunktion des Subnetzwerks implementiert und/oder komplementäre Inputsignale verwendet, ist das **Polarity Assignment Problem**.

Beispiel

Jede Clusterfunktion $f(a, b)$ in der Menge:

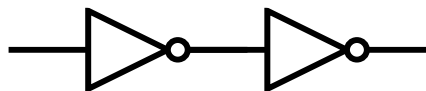
$$\{a \vee b, \bar{a} \vee b, a \vee \bar{b}, \bar{a} \vee \bar{b}, ab, \bar{a}b, a\bar{b}, \bar{a}\bar{b}\}$$

kann durch die Pattern Funktion $g(x, y) = x \vee y$ (OR2-Zelle) mit verschiedenen Kombinationen invertierter Eingänge/Ausgänge gematched werden.

Polarity Assignment Problem

Structural Covering

- Für das **Structural Covering** können alle Verbindungen zwischen basis Gates durch ein **Paar Inverter** ersetzt werden, was das Netzwerkverhalten nicht beeinträchtigt.
- Durch Hinzufügen einer **fake Bibliothekszelle**, die zwei Invertern in Reihe mit einer **direkten Verbindung** und **Kosten von Null** entspricht, kann der Structural Matching Algorithmus entweder die *virtuellen Inverter* nutzen und sie in Zellen mit geringeren Gesamtkosten integrieren oder sie durch eine einfache Verbindung mit Kosten von Null ersetzen.

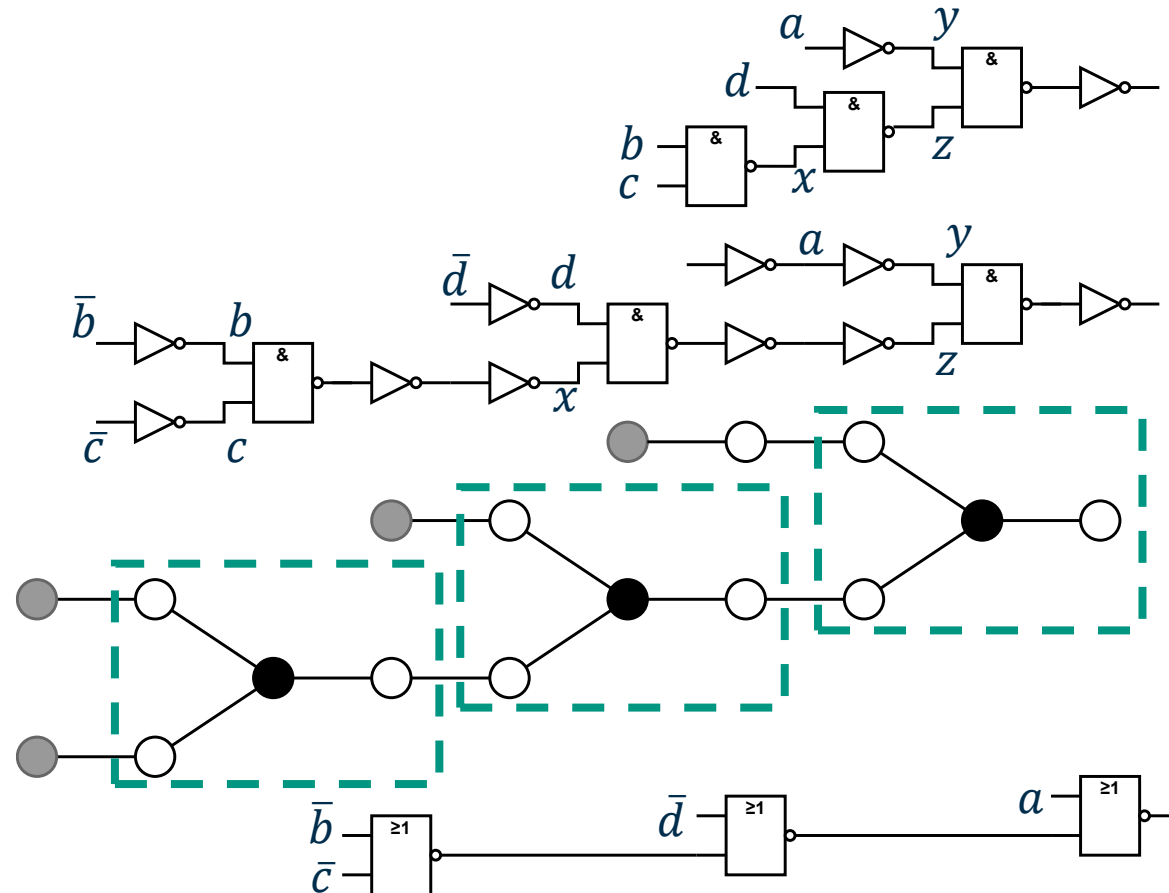


Polarity Assignment Problem

Structural Covering

Beispiel Structural Covering

- Ursprüngliches Netzwerk
- Netzwerk nach Einfügen von Inverterpaaren
- Subject Tree
- Netzwerkabdeckung





Rule-Based Library Binding

5

Rule-Based Library Binding

Einführung

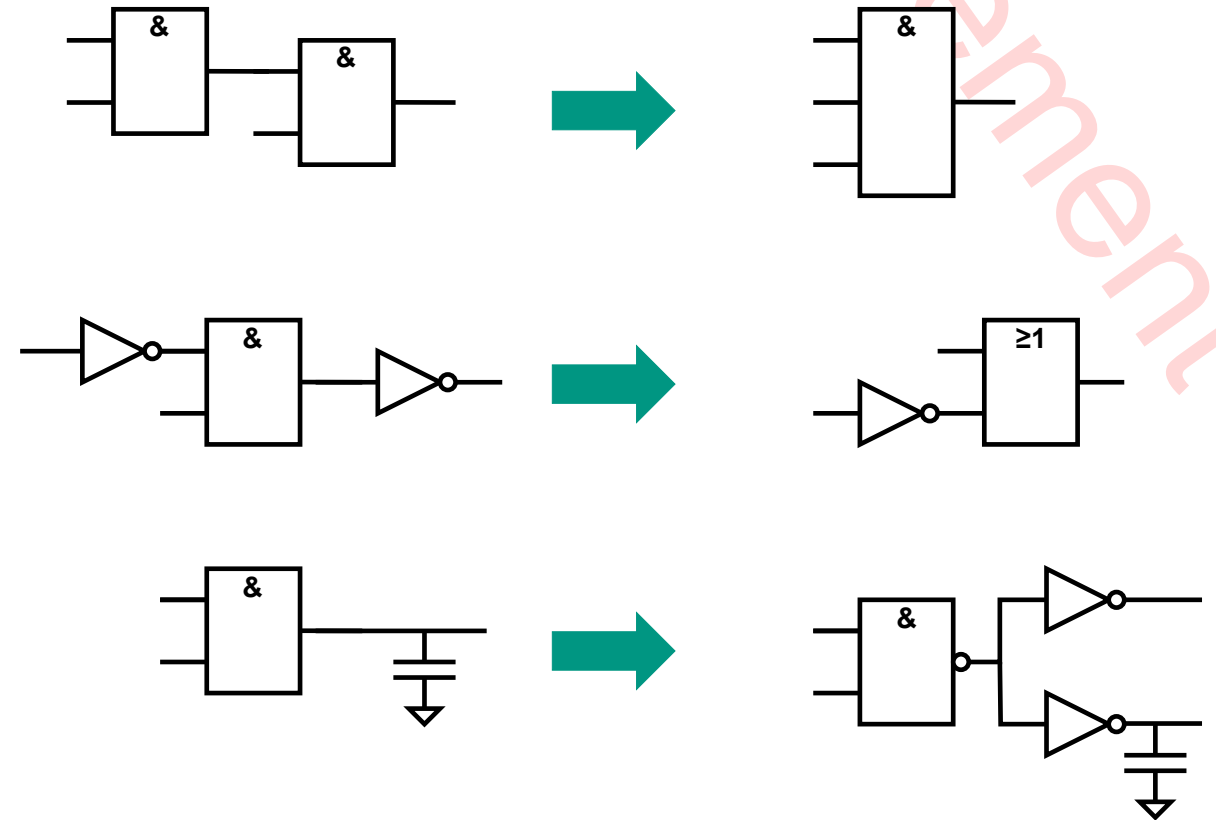
- Rule-based library binding ist eine Alternative und Ergänzung zu den algorithmischen Ansätzen.
- Ein Netzwerk wird durch schrittweise Verfeinerung gecovert.
 - Eine einfache Regel schlägt die beste Übereinstimmung für ein Subnetzwerk vor.
 - Eine komplexe Regel schlägt eine Umstrukturierung des Netzwerks vor.

Rule-Based Library Binding

Einführung

Beispiel 3 Regeln (einfach + 2x komplex)

1. Zwei kaskadierte two-input AND-Gatter können an ein three-input AND-Gatter geconvertet werden.
2. Two-input AND-Gatter mit invertiertem Input/Output kann mit einem two-input NOR-Gatter mit invertiertem Input geconvertet werden.
3. Ein UND-Gatter auf einem kritischen Pfad mit hoher Last kann durch ein NAND-Gatter ersetzt werden, dem zwei Inverter parallel folgen.



Rule-Based Library Binding

Einführung

- Der Ansatz geht von einem **Trivial Bound Netzwerk** aus und wendet Transformationen darauf an.
- Die Ausführung der Regeln folgt einem **Prioritätsschema**.
 - Für jede Regel durchsucht der Algorithmus das Netzwerk nach einem Pattern und wendet die entsprechende Transformation an.
 - Üblicherweise kommen **Greedy Search** oder fortgeschrittenere Suchverfahren zum Einsatz, die verschiedene Optionen untersuchen und die beste Transformation anwenden.



Zieltechnologie CMOS

6

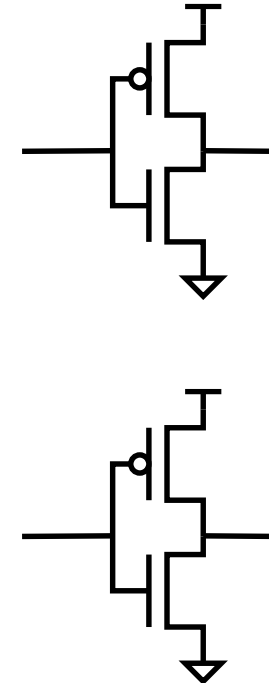
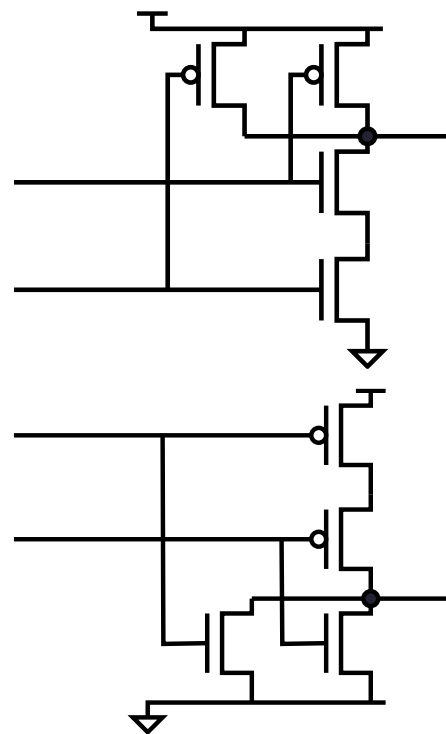
Zieltechnologie CMOS

Einführung

- Alle booleschen Funktionen können auf ein Basissystem logischer Schaltungen zurückgeführt werden.
- Bei der CMOS-Technologie sind zwei Basissysteme üblich, da die Grundzellen einfach zu implementieren sind:

- NAND2 / NOT-Basissystem

- NOR2 / NOT-Basissystem



Zieltechnologie CMOS

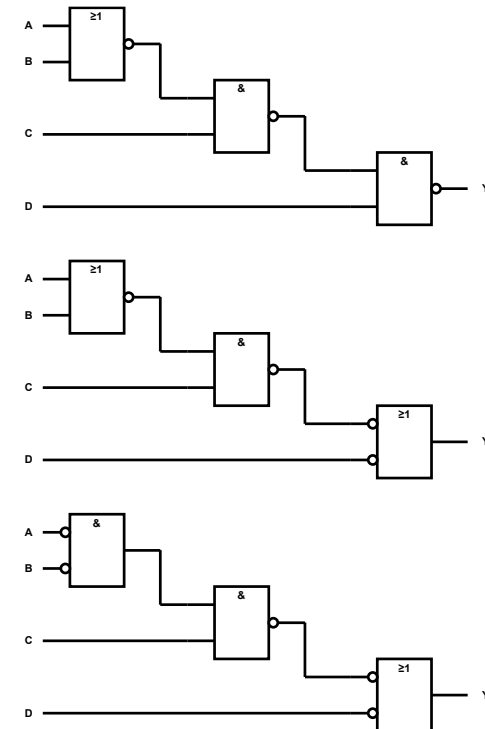
Einführung

Anmerkung

Bubble Pushing hilft dabei, die Negationen in solchen Basiszellsystemen mit negierten Ein-/Ausgängen zu analysieren.

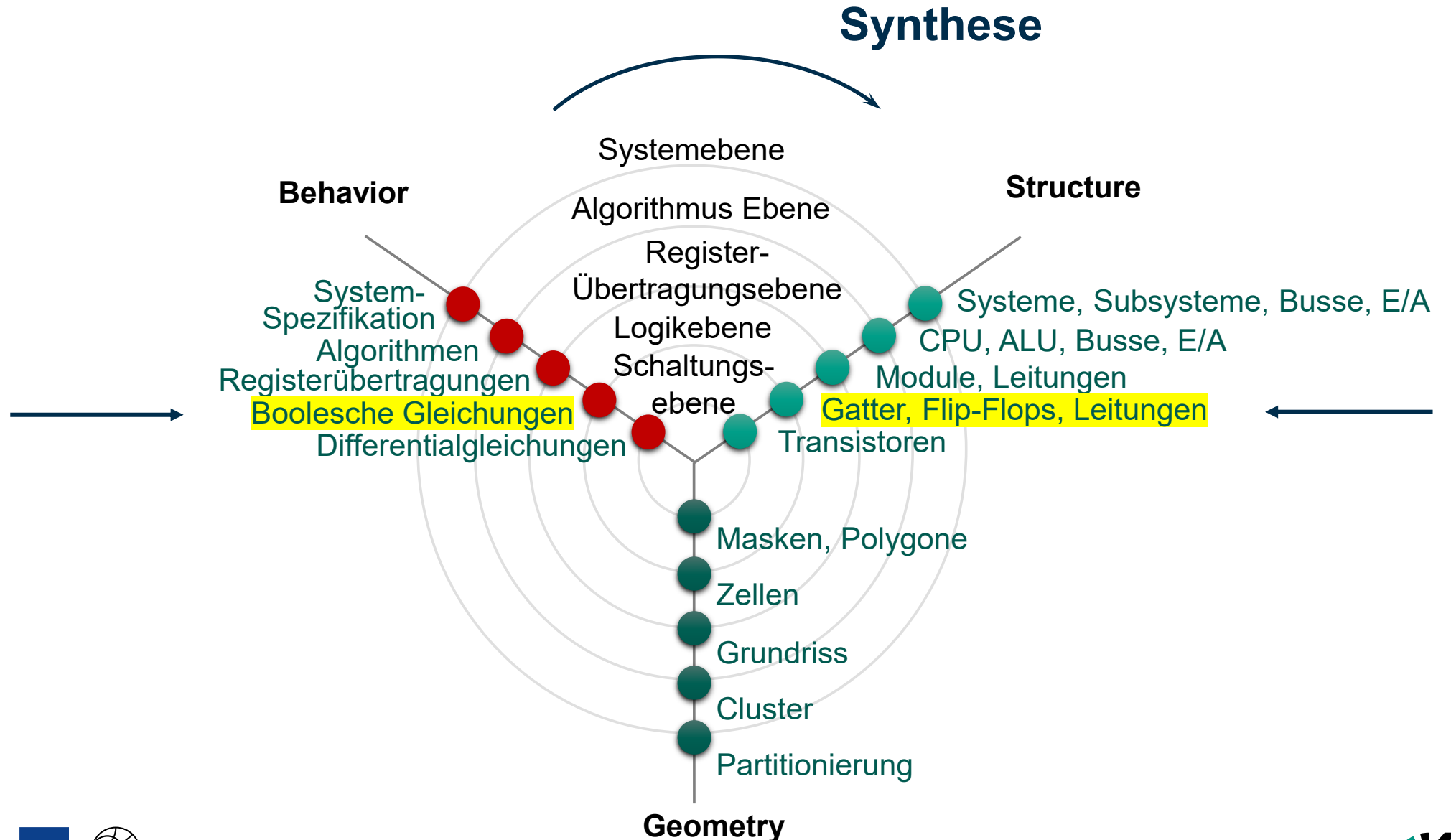
Siehe Vorlesung

12 Kombinatorische Logikschaltungen.



Advance Organizer

Y-Diagramm



Referenz

- [1] S. L. Harris und D. M. Harris, *Digitales Design und Computerarchitektur*, ARM®-Ausgabe. Waltham, MA: Morgan Kaufmann, 2016.
- [2] G. De Micheli, *Lösungshandbuch zur Synthese und Optimierung digitaler Schaltungen*. New York: McGraw-Hill, 1995.
- [3] R. J. Francis, *Technologie-Mapping für Lookup-Tabellen-basierte feldprogrammierbare Gate-Arrays*. Kanada, Universität Toronto, 1993.

